Aarya Arban

T11 – 05

Assignment No. 8

Aim: To implement classification modelling.

Theory:

Classification modelling is a supervised learning technique used to categorize data into predefined classes or categories. The goal is to build a model that can accurately predict the class label of new, unseen data points based on the patterns learned from the training data. Several algorithms can be employed for classification, and understanding their underlying principles is crucial for effective model building and evaluation.

Here's a more detailed explanation of the algorithms mentioned and the performance metrics used in classification:

1. **Logistic Regression:**

Logistic Regression is a linear model primarily used for binary classification problems, where the target variable has only two possible outcomes (e.g., yes/no, spam/not spam, 0/1). Despite its name, it's a classification algorithm, not a regression algorithm.

- **Model Representation:** Logistic Regression models the probability of an instance belonging to a specific class (typically the positive class, labelled as 1). It uses a linear combination of the input features, but instead of directly outputting the value, it applies a sigmoid function (also known as the logistic function) to the linear combination.
- **Hypothesis Function:** The hypothesis function in Logistic Regression is given by: $h\theta(x)=\sigma(\theta Tx)=1+e-\theta Tx1$ where:
    - $h\theta(x)$ is the predicted probability of the instance belonging to the positive class. θ represents the model parameters (coefficients).
    - x is the vector of input features. $\sigma(z)=1+e-z1$ is the sigmoid function, which squashes any real value into the range [0, 1].
- **Decision Boundary:** The logistic regression model predicts the class based on whether the predicted probability exceeds a certain threshold (commonly 0.5). If $h\theta(x)\geq 0.5$, the instance is predicted as belonging to the positive class (1); otherwise, it's predicted as belonging to the

negative class (0). The decision boundary is the line or hyperplane where hθ(x)=0.5, which corresponds to θTx=0.

- **Training:** The model parameters θ are learned from the training data by minimizing a cost function. The most common cost function used in Logistic Regression is the

  logistic loss (also known as cross-entropy loss):

  <1>J(θ)=−m1∑i=1m[y(i)log(hθ (x(i)))+(1−y(i))log(1−hθ(x(i)))] where: o

  m is the number of training examples. o y(i) is the actual class

  label of the i-th training example (0 or 1).

  - hθ(x(i)) is the predicted probability for the i-th training example. Optimization algorithms like **Gradient Descent** or its variants (e.g., Stochastic Gradient Descent) are used to find the values of θ that minimize this cost function.

- **Prediction:** Once the model is trained, for a new input instance, the probability of it belonging to the positive class is calculated using the hypothesis function. This probability is then compared to the chosen threshold to assign the final class label.

2. Decision Tree:

Decision Trees are versatile non-parametric supervised learning algorithms that can be used for both classification and regression tasks. They work by recursively partitioning the feature space into regions based on feature values.

- **Tree Construction:** The process of building a decision tree involves selecting the best feature to split the data at each node. This selection is based on criteria that aim to maximize the separation of classes (for classification) or reduce variance (for regression). Common splitting criteria for classification include:

  - **Gini Impurity:** Measures the impurity of a set of instances. A Gini impurity of

  0 indicates perfect purity (all instances belong to the same class). o

  Information Gain: Measures the reduction in entropy (a measure of

  disorder) achieved by splitting the data on a particular feature.

  - **Chi-squared:** Measures the statistical significance of the difference between the observed and expected frequencies of the target variable across different categories of the feature.

- **Recursive Partitioning:** The splitting process continues recursively for each resulting subset until a stopping criterion is met. These criteria can include: ○ **Maximum Depth:** Limits the depth of the tree to prevent overfitting. ○ **Minimum Samples per Leaf:** Ensures that each leaf node contains a minimum number of data points.
    - **Purity Threshold:** Stops splitting if a node becomes sufficiently pure (e.g., all instances belong to the same class).
- **Prediction:** To classify a new instance, the algorithm starts at the root node of the tree and traverses down the tree based on the values of the instance's features. At each internal node, a test is performed on a specific feature, and the instance follows the branch corresponding to the outcome of the test. This process continues until a leaf node is reached. The leaf node assigns the class label based on the majority class of the training instances that fall into that leaf.

Performance Metrics for Classification:

Evaluating the performance of a classification model is crucial to understand how well it generalizes to unseen data. Several metrics can be used, each providing different insights into the model's capabilities.

1. **Accuracy:**
    - **Definition:** The proportion of correctly classified instances out of the total number of instances. ○ **Formula:** Accuracy = (Number of Correct Predictions) / (Total Number of Instances) ○ **Pros:** Simple to understand and calculate.
    - **Cons:** Can be misleading when dealing with imbalanced datasets, where one class has significantly more instances than the others. A model can achieve high accuracy by simply predicting the majority class.
2. **Precision:**
    - **Definition:** The proportion of true positive predictions (correctly predicted positive instances) out of all instances predicted as positive. It measures the model's ability to avoid false positives. ○ **Formula:** Precision = True Positives (TP) / (True Positives (TP) + False Positives (FP)) ○ **Interpretation:** Of all the instances the model predicted as positive, what proportion were actually positive?
3. **Recall (Sensitivity, True Positive Rate):**

- **Definition:** The proportion of true positive predictions out of the total number of actual positive instances. It measures the model's ability to find all the positive instances. ₒ **Formula:** Recall = True Positives (TP) / (True
  Positives (TP) + False Negatives (FN))

  Interpretation: Of all the instances that were actually positive, what proportion did the model correctly identify?

4. **F1 Score:**

   - **Definition:** The harmonic mean of precision and recall. It provides a balanced measure of the model's performance, especially useful when precision and recall have conflicting values. ₒ **Formula:** F1 Score = 2 * (Precision *
     Recall) / (Precision + Recall)

   - **Interpretation:** A high F1 score indicates that the model has both good precision and good recall.

5. **ROC Curve and AUC (Area Under the ROC Curve):**

   - **ROC Curve:** A graphical plot that illustrates the diagnostic ability of a binary classifier as its discrimination threshold is varied. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different threshold settings.

     - **TPR (Sensitivity):** TP / (TP + FN)
     - **FPR:** FP / (FP + TN)

   - **AUC:** Represents the area under the ROC curve. It provides a single scalar value that summarizes the overall performance of the classifier. An AUC of 0.5 indicates random guessing, while an AUC of 1 indicates a perfect classifier.

   - **Interpretation:** A higher AUC generally indicates a better classifier. The ROC curve and AUC are particularly useful when you need to understand the trade-off between true positives and false positives.

6. **Confusion Matrix:**

- **Definition:** A table that summarizes the performance of a classification model by displaying the counts of true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions.
- **Structure (for binary classification):**

7. Here's the formatted confusion matrix table:

Actual Class \ Predicted Class Positive (1) Negative (0)

Positive (1) (Actual) **TP** (True Positive) **FN** (False Negative) **Negative (0)** (Actual) **FP** (False Positive) **TN** (True Negative)

- **Interpretation:** The confusion matrix provides a detailed breakdown of the model's predictions, allowing for a more nuanced evaluation than simple accuracy. It helps identify the types of errors the model is making.

Heart Disease UCI dataset

1. **age**:
    - The age of the patient. This is a continuous variable that indicates the person's age at the time of data collection.
2. **sex**:
    - The gender of the patient, where:
        - `1` = Male
        - `0` = Female
3. **cp** (Chest Pain Type):
    - This variable indicates the type of chest pain the patient is experiencing, and it is typically encoded with numerical values that represent different types of chest pain:
        - `0` = Typical Angina
        - `1` = Atypical Angina
        - `2` = Non-anginal Pain
        - `3` = Asymptomatic (no pain)
4. **trestbps** (Resting Blood Pressure):
    - The blood pressure of the patient when they are at rest, measured in mmHg (millimeters of mercury).
5. **chol** (Serum Cholesterol):
    - The level of serum cholesterol in mg/dl (milligrams per deciliter). Cholesterol levels can be an important indicator of heart disease risk.
6. **fbs** (Fasting Blood Sugar):
    - A binary variable that indicates whether the patient's fasting blood sugar level is greater than 120 mg/dl.
        - `1` = Fasting blood sugar > 120 mg/dl
        - `0` = Fasting blood sugar <= 120 mg/dl
7. **restecg** (Resting Electrocardiographic Results):
    - This variable represents the electrocardiographic results at rest, which is typically used to assess heart activity:
        - `0` = Normal
        - `1` = Having ST-T wave abnormality (could suggest a problem with heart muscle or coronary artery disease)
        - `2` = Showing probable or definite left ventricular hypertrophy (LVH), which is an enlargement of the heart's left ventricle.

8. **thalach** (Maximum Heart Rate Achieved):
    - The maximum heart rate achieved during exercise. Higher heart rates during physical exertion may indicate heart health or potential cardiovascular problems.
9. **exang** (Exercise Induced Angina):
    - A binary variable indicating whether the patient experienced angina (chest pain) during exercise:
        - `1` = Yes (Exercise induced angina)
        - `0` = No (No exercise induced angina)
10. **oldpeak**:
    - This variable refers to depression induced by exercise relative to rest. It's often used to assess how much the ST segment of the ECG is lowered during physical activity, which can be a sign of heart disease.
    - `oldpeak` can be a continuous numerical value representing this depression (i.e., how much the ST segment is displaced).
11. **slope** (Slope of the Peak Exercise ST Segment):
    - The slope of the ST segment in the electrocardiogram after exercise, which is an important indicator of heart health:
        - `0` = Upsloping
        - `1` = Flat
        - `2` = Downsloping
12. **ca** (Number of Major Vessels Colored by Fluoroscopy):
    - This variable indicates the number of major coronary arteries that are visible with fluoroscopy (a type of X-ray), which can help detect blockages:
        - `0` = No vessels colored by fluoroscopy (i.e., no visible blockages)
        - `1` = One vessel blocked
        - `2` = Two vessels blocked
        - `3` = Three vessels blocked
13. **thal** (Thalassemia):
    - A blood disorder that can affect the heart. It is an important variable for diagnosing heart disease, and typically has the following values:
        - `1` = Normal
        - `2` = Fixed defect (indicating a part of the heart muscle doesn't get blood during exercise)
        - `3` = Reversible defect (indicating a part of the heart muscle temporarily doesn't get blood during exercise)

14. **target**:
    - This is the target variable you are trying to predict. It indicates whether the patient has heart disease:
        - `1` = The patient has heart disease.
        - `0` = The patient does not have heart disease.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


# loading the csv data to a Pandas DataFrame
heart_data = pd.read_csv('/content/heart_disease_data.csv')


# print first 5 rows of the dataset
heart_data.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	tha
0	63	1	3	145	233	1	0	150	0	2.3	0	0	
1	37	1	2	130	250	0	1	187	0	3.5	0	0	
2	41	0	1	130	204	0	0	172	0	1.4	2	0	
3	56	1	1	120	236	0	1	178	0	0.8	2	0	
4	57	0	0	120	354	0	1	163	1	0.6	2	0	

```
# number of rows and columns in the dataset
heart_data.shape
```

```
(303, 14)
```

```
# getting some info about the data
heart_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
```

```
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
# checking for missing values
heart_data.isnull().sum()
```

	0
age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

dtype: int64

```
# statistical measures about the data
heart_data.describe()
```

	age	sex	cp	trestbps	chol	fbs	restec
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.00000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.52805
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.52586
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.00000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.00000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.00000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.00000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.00000

```
# checking the distribution of Target Variable
heart_data['target'].value_counts()
```

target	count
1	165
0	138

dtype: int64

```
X = heart_data.drop(columns='target', axis=1)
Y = heart_data['target']
```

```
print(Y)
```

```
0      1
1      1
2      1
3      1
4      1
      ..
298    0
299    0
300    0
301    0
302    0
Name: target, Length: 303, dtype: int64
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, stratify=Y, rand
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(303, 13) (242, 13) (61, 13)
```

```
model = LogisticRegression()
```

```
# training the LogisticRegression model with Training data
model.fit(X_train, Y_train)
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: Conver
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

LogisticRegression

LogisticRegression()

```
# accuracy on training data
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)
```

```
print('Accuracy on Training data : ', training_data_accuracy)
```

```
Accuracy on Training data :  0.8512396694214877
```

```
# accuracy on test data
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)
```

```
print('Accuracy on Test data : ', test_data_accuracy)
```

```
Accuracy on Test data :  0.819672131147541
```

Start coding or generate with AI.

Start coding or generate with AI.

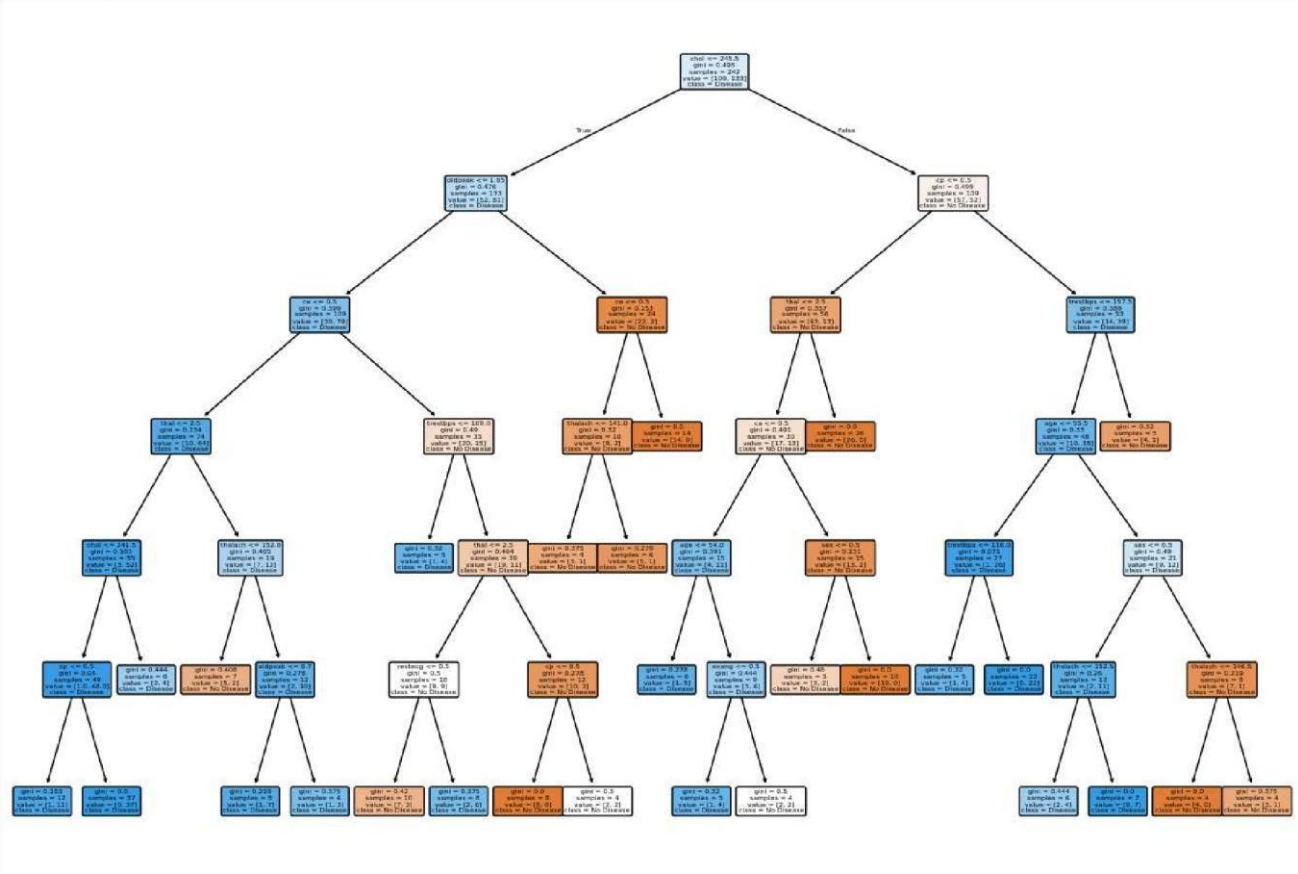
```
input_data = (62,0,0,140,268,0,0,160,0,3.6,0,2,2)

# change the input data to a numpy array
input_data_as_numpy_array= np.asarray(input_data)

# reshape the numpy array as we are predicting for only on instance
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

prediction = model.predict(input_data_reshaped)
print(prediction)

if (prediction[0]== 0):
  print('The Person does not have a Heart Disease')
else:
  print('The Person has Heart Disease')
```

```
[0]
The Person does not have a Heart Disease
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning
  warnings.warn(
```

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics
from sklearn.metrics import confusion_matrix
import numpy as np
import matplotlib.pyplot as plt
from sklearn import tree
```

```
# Define the model
clf = DecisionTreeClassifier(random_state=42)

# Define hyperparameters to tune
param_grid = {
    'criterion': ['gini'],
    'max_depth': [6],
    'min_samples_split': [2],
    'min_samples_leaf': [4],
    'max_features': [None, 'sqrt', 'log2']
}

grid_search = GridSearchCV(estimator=clf, param_grid=param_grid, cv=5, n_jobs=-1, verbose

grid_search.fit(X_train, y_train)

print(f"Best Hyperparameters: {grid_search.best_params_}")

best_clf = grid_search.best_estimator_

y_pred = best_clf.predict(X_test)

accuracy = metrics.accuracy_score(y_test, y_pred)
print(confusion_matrix(y_test, y_pred))
print(f"Accuracy: {accuracy * 100:.2f}%")

plt.figure(figsize=(15, 10))
tree.plot_tree(best_clf, filled=True, feature_names=X.columns, class_names=["No Disease",
plt.show()
```

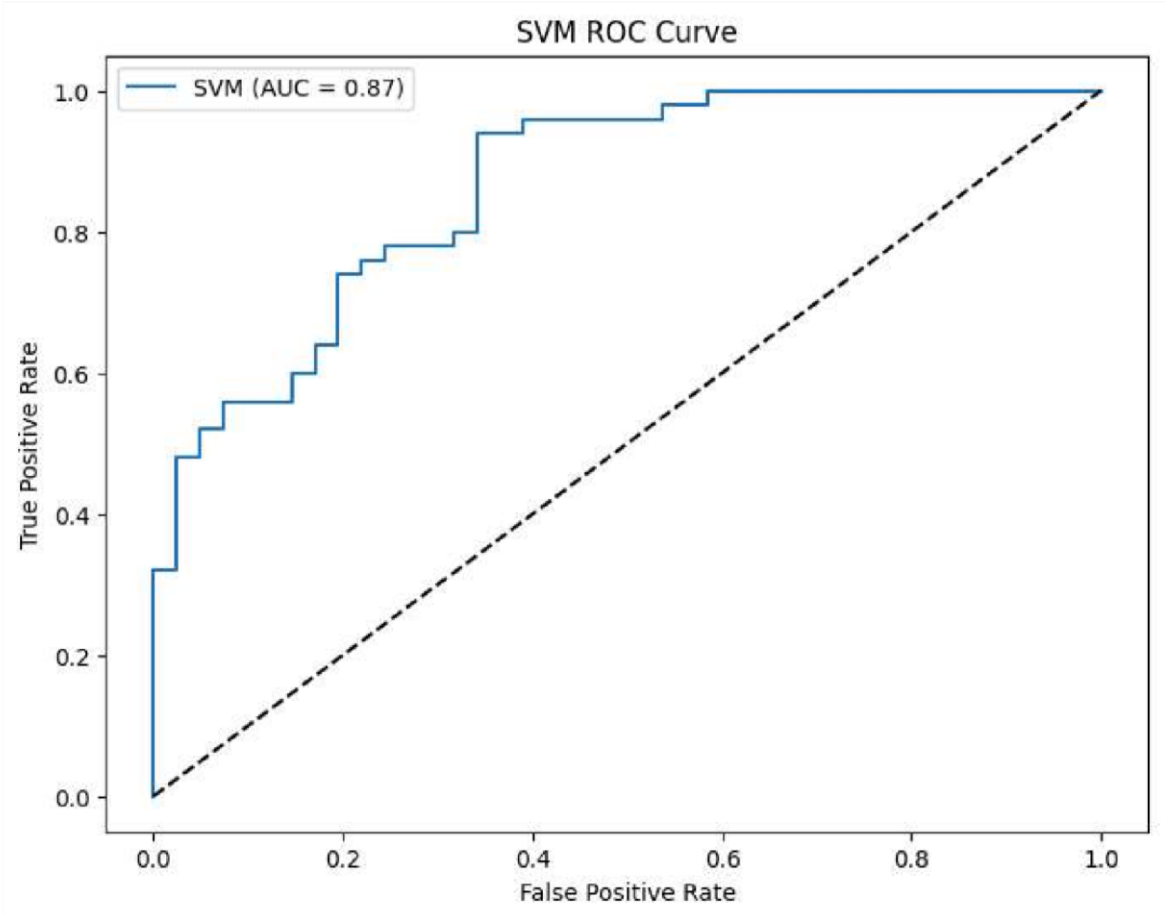
```
Fitting 5 folds for each of 3 candidates, totalling 15 fits
Best Hyperparameters: {'criterion': 'gini', 'max_depth': 6, 'max_features': 'sqrt', '
[[26  3]
 [ 8 24]]
Accuracy: 81.97%
```

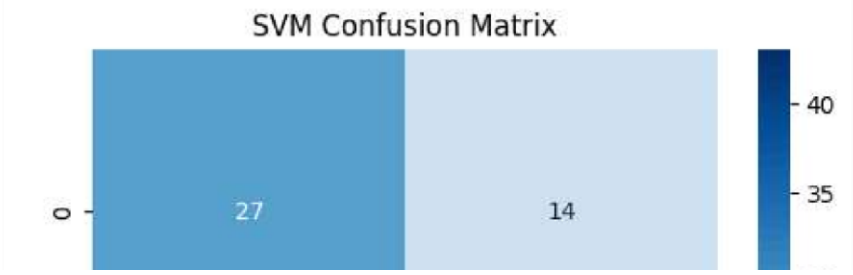


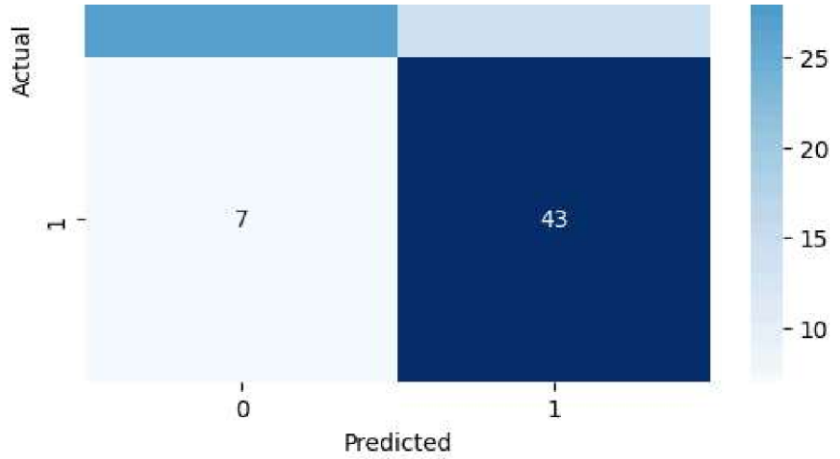
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, accuracy_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import seaborn as sns


try:
    df = pd.read_csv('/content/heart.csv')
    X = df.drop('target', axis=1)
    y = df['target']
except FileNotFoundError:
    print("Error: heart.csv not found in the current directory. Please make sure the file is in the correct l
    exit()


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)


def tune_clf_hyperparameters(pipeline, param_grid, X_train, y_train, scoring='accuracy', cv=3):
    """
    Tunes the hyperparameters of a classifier using GridSearchCV.

    Args:
        pipeline: The scikit-learn pipeline containing the classifier.
        param_grid: Dictionary of hyperparameters to tune.
        X_train: Training features.
        y_train: Training target.
        scoring: Scoring metric for GridSearchCV (e.g., 'accuracy', 'f1', 'roc_auc').
        cv: Number of cross-validation folds.

    Returns:
        best_model: The best performing model from GridSearchCV.
        best_params: The hyperparameters of the best model.
    """
    grid_search = GridSearchCV(pipeline, param_grid, cv=cv, scoring=scoring, verbose=2, n_jobs=-1)
    grid_search.fit(X_train, y_train)
    best_params = grid_search.best_params_
    best_model = grid_search.best_estimator_
    return best_model, best_params


def evaluate_model(model, X_test, y_test, model_name='Model'):
    """
    Evaluates a classification model on the test set and prints performance metrics.

    Args:
        model: Trained classification model.
        X_test: Test features.
        y_test: Test target.
        model_name: Name of the model for reporting.

    Returns:
        evaluation_dict: Dictionary containing evaluation metrics.
    """
    y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred)
```

```
    print(f"--- {model_name} Evaluation ---")
    print(f"Accuracy: {accuracy:.4f}")
    print("Classification Report:\n", report)
    print(f"AUC: {auc:.4f}")

    fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, label=f'{model_name} (AUC = {auc:.2f})')
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'{model_name} ROC Curve')
    plt.legend()
    plt.show()

    # Plot Confusion Matrix
    cm = pd.crosstab(y_test, y_pred, rownames=['Actual'], colnames=['Predicted'])
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f'{model_name} Confusion Matrix')
    plt.show()

    evaluation_dict = {
        'model': model_name,
        'accuracy': accuracy,
        'classification_report': report,
        'auc': auc
    }
    return evaluation_dict


svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(probability=True, random_state=42))  # Added random_state for reproducibility
])

param_grid_svm = {
    'svm__C': [0.001, 0.01, 0.1, 1, 10],
    'svm__kernel': ['linear', 'rbf'],
    'svm__gamma': ['scale', 'auto', 0.1, 1]
    # Removed 'poly' and 'degree' for simplicity, but you can include them if needed
    # 'svm__degree': [2, 3]
}

best_svm, best_svm_hyperparams = tune_clf_hyperparameters(svm_pipeline, param_grid_svm, X_train, y_train, sc
print('SVM Optimal Hyperparameters: \n', best_svm_hyperparams)


print("\n--- SVM Evaluation on Training Data ---")
print(classification_report(y_train, best_svm.predict(X_train)))


print("\n--- SVM Evaluation on Test Data ---")
svm_evaluation = evaluate_model(best_svm, X_test, y_test, 'SVM')
print(svm_evaluation)
```

```
Fitting 3 folds for each of 40 candidates, totalling 120 fits
SVM Optimal Hyperparameters:
 {'svm__C': 0.1, 'svm__gamma': 'scale', 'svm__kernel': 'linear'}

--- SVM Evaluation on Training Data ---
              precision    recall  f1-score   support

           0       0.92      0.81      0.86        97
           1       0.86      0.94      0.90       115

    accuracy                           0.88       212
   macro avg       0.89      0.88      0.88       212
weighted avg       0.89      0.88      0.88       212


--- SVM Evaluation on Test Data ---
--- SVM Evaluation ---
Accuracy: 0.7692
Classification Report:
               precision    recall  f1-score   support

           0       0.79      0.66      0.72        41
           1       0.75      0.86      0.80        50

    accuracy                           0.77        91
   macro avg       0.77      0.76      0.76        91
weighted avg       0.77      0.77      0.77        91

AUC: 0.8654
```







{'model': 'SVM', 'accuracy': 0.7692307692307693, 'classification_report': ' precision

Start coding or generate with AI.

Conclusion:

Understanding the principles of classification algorithms like Logistic Regression and Decision Trees, along with the appropriate performance metrics, is essential for building effective classification models. The choice of algorithm and the metrics used for evaluation depend on the specific problem, the characteristics of the data, and the desired trade-offs between different types of errors. By carefully selecting and evaluating classification models, we can gain valuable insights and make accurate predictions on categorical data.